

Universidad de San Carlos de Guatemala

Facultad de Ingeniería

Escuela de Ciencias y Sistemas

Bases de Datos 1



MANUAL DE USUARIO

OxygenScript

Emily Maritza Tepeu Guacamaya

202402955

Proyecto 1

CONTENIDO

INTRODUCCIÓN	3
INSTALACIÓN	4
Requisitos previos	4
Clonar el Repositorio	4
Levantar el Backend	4
Levantar el Frontend	5
PARTES DE LA INTERFAZ	6
Crear, abrir y guardar un archivo	6
EJECUTAR PROGRAMA	7
Programa compilado	7
REPORTE DE ERRORES	8
Error típico	8
Programa compilado con errores	9
TABLA DE SÍMBOLOS	10
Shadowing	10
¿Qué pasa si el código tiene errores?	11
ÁRBOL DE SINTAXIS ABSTRACTA	12
PREGUNTAS FRECUENTES	13
¿Por qué el programa no imprime nada?	13
¿Se puede usar for?	13
¿Cómo le pongo nombre a un loop para poder hacerle break/continue desde uno anidado?	13
¿Por qué un String con \n no se ve en dos líneas en la consola?	13

INTRODUCCIÓN

OxygenScript es un lenguaje con una sintaxis inspirada en Rust de tipado estático que soporta variables mutables e inmutables, estructuras de control (if, while, loop, match), arreglos, slices, cadenas de texto, structs, funciones con parámetros y valor de retorno, y funciones embebidas para trabajar con texto y colecciones.

Este proyecto implementa un intérprete completo para OxygenScript, un programa capaz de leer código fuente escrito en el lenguaje, analizarlo léxica, sintáctica y semánticamente, y ejecutarlo mostrando su salida en tiempo real. El intérprete se ejecuta a través de una interfaz web, donde es posible escribir o cargar código, ejecutarlo, y revisar los reportes generados automáticamente que son los errores detectados en cada fase del análisis, la tabla de símbolos con las variables, funciones y structs declarados, y el árbol de sintaxis abstracta AST del programa.

INSTALACIÓN

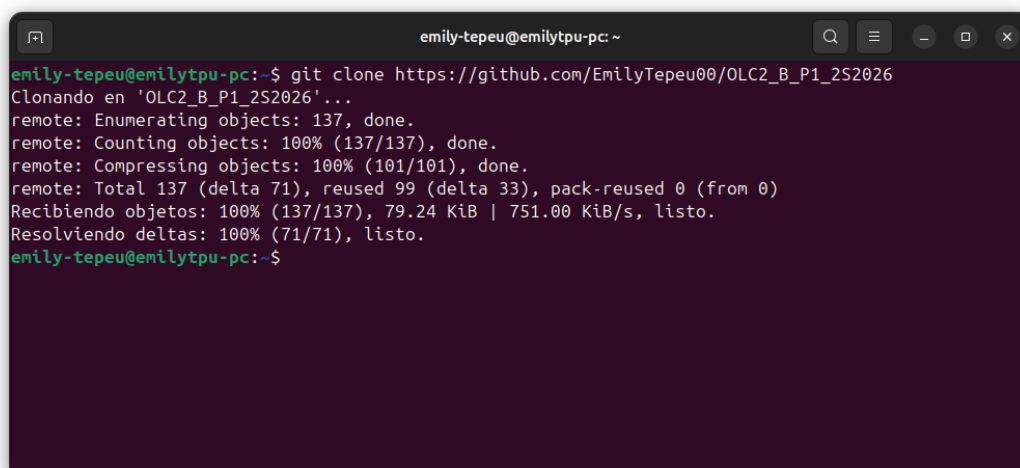
Requisitos previos

- Python 3.10+
- Node.js 18+
- Graphviz instalado a nivel de sistema operativo
 - Linux Ubuntu/Debian: `sudo apt install graphviz`

Clonar el Repositorio

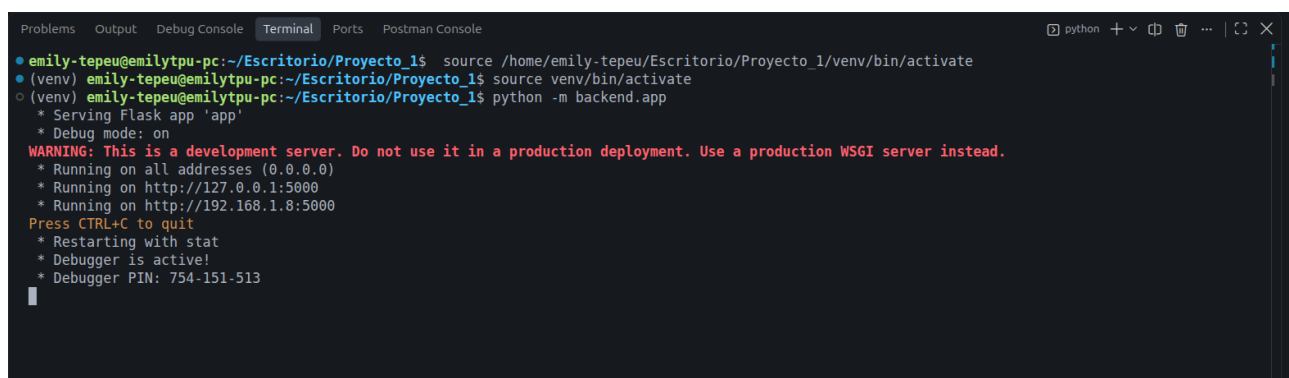
`git clone https://github.com/EmilyTepeu00/OLC2_B_P1_2S2026`

`cd OLC2_B_P1_2S2026`



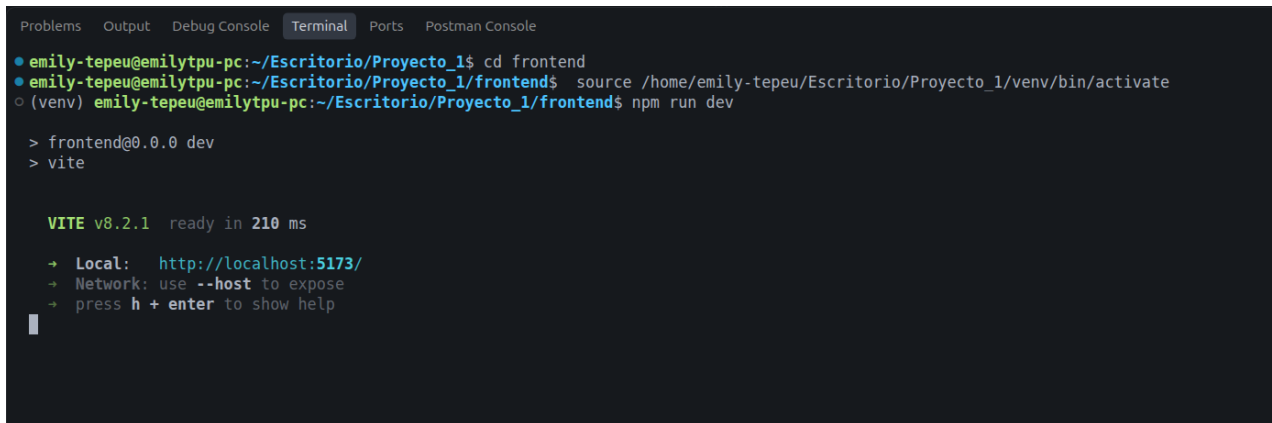
```
emily-tepeu@emilytpu-pc: ~  
emily-tepeu@emilytpu-pc:~$ git clone https://github.com/EmilyTepeu00/OLC2_B_P1_2S2026  
Clonando en 'OLC2_B_P1_2S2026'...  
remote: Enumerating objects: 137, done.  
remote: Counting objects: 100% (137/137), done.  
remote: Compressing objects: 100% (101/101), done.  
remote: Total 137 (delta 71), reused 99 (delta 33), pack-reused 0 (from 0)  
Recibiendo objetos: 100% (137/137), 79.24 KiB | 751.00 KiB/s, listo.  
Resolviendo deltas: 100% (71/71), listo.  
emily-tepeu@emilytpu-pc:~$
```

Levantar el Backend



```
Problems Output Debug Console Terminal Ports Postman Console  
emily-tepeu@emilytpu-pc:~/Escritorio/Proyecto_1$ source /home/emily-tepeu/Escritorio/Proyecto_1/venv/bin/activate  
(venv) emily-tepeu@emilytpu-pc:~/Escritorio/Proyecto_1$ source venv/bin/activate  
(venv) emily-tepeu@emilytpu-pc:~/Escritorio/Proyecto_1$ python -m backend.app  
* Serving Flask app 'app'  
* Debug mode: on  
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.  
* Running on all addresses (0.0.0.0)  
* Running on http://127.0.0.1:5000  
* Running on http://192.168.1.8:5000  
Press CTRL+C to quit  
* Restarting with stat  
* Debugger is active!  
* Debugger PIN: 754-151-513
```

Levantar el Frontend



```
Problems  Output  Debug Console  Terminal  Ports  Postman Console
• emily-tepeu@emilytpu-pc:~/Escritorio/Proyecto_1$ cd frontend
• emily-tepeu@emilytpu-pc:~/Escritorio/Proyecto_1/frontend$ source /home/emily-tepeu/Escritorio/Proyecto_1/venv/bin/activate
○ (venv) emily-tepeu@emilytpu-pc:~/Escritorio/Proyecto_1/frontend$ npm run dev

> frontend@0.0.0 dev
> vite

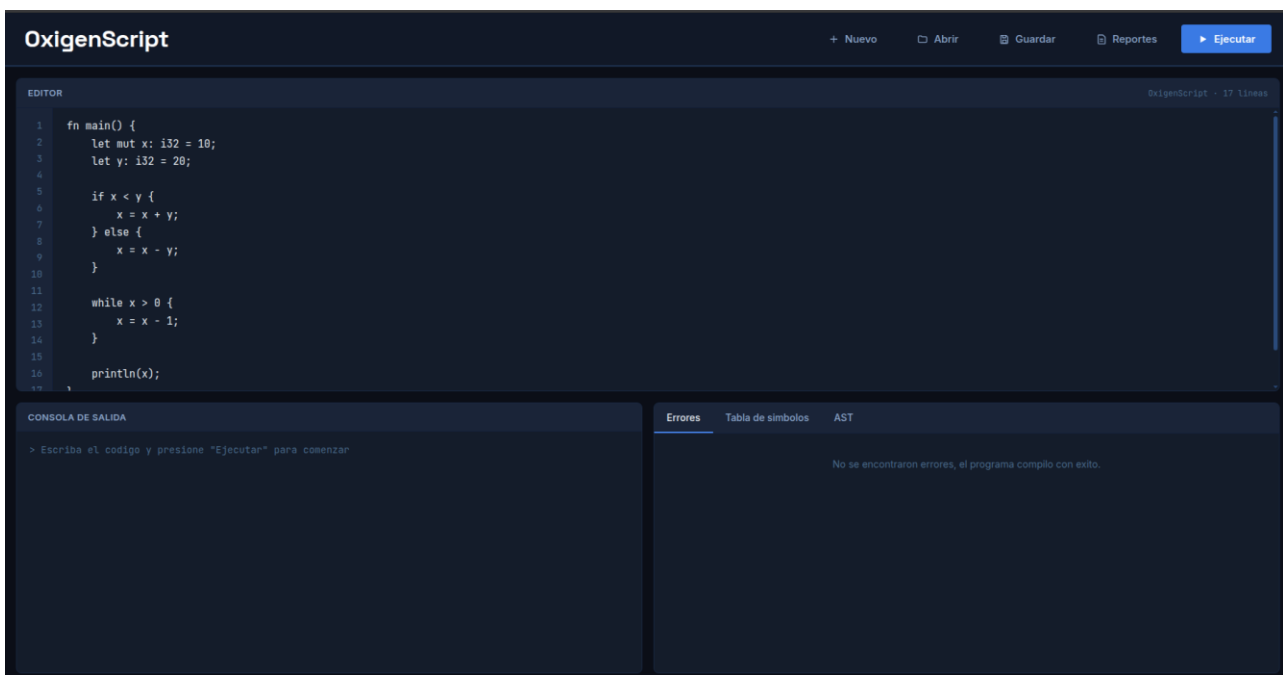
VITE v8.2.1  ready in 210 ms
➔ Local:   http://localhost:5173/
➔ Network: use --host to expose
➔ press h + enter to show help
```

Por defecto queda corriendo en <http://localhost:5173>, justo como se muestra en la terminal.

PARTES DE LA INTERFAZ

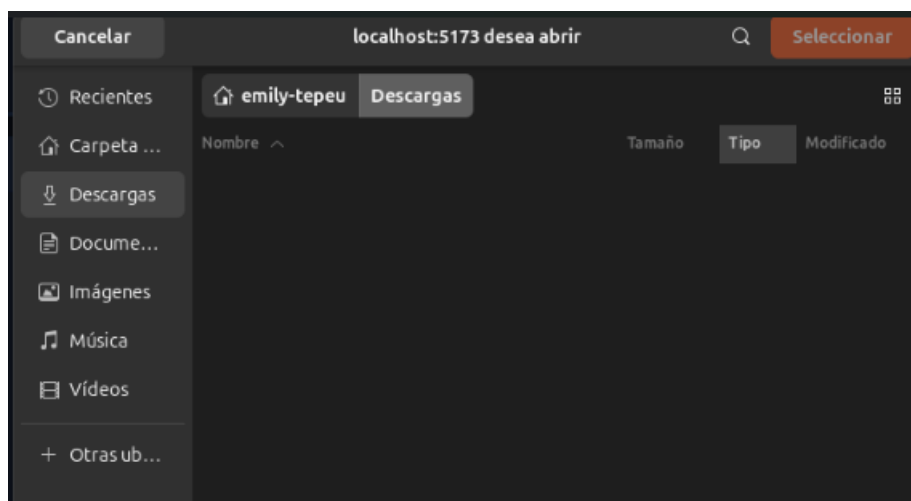
La interfaz muestra la pantalla principal de OxygenScript la cual cuenta con las siguientes funciones:

- Barra de herramientas: botones Nuevo, Abrir, Guardar y Ejecutar.
- Editor de código: para escribir y pegar el código del programa.
- Consola de salida: muestra el resultado de la ejecución, línea por línea.
- Panel de reportes: muestra Errores, Tabla de símbolos y AST.



Crear, abrir y guardar un archivo

- Nuevo: limpia el editor de código para iniciar de cero.
- Abrir: abre el selector de archivos para elegir un archivo de extensión .ox, .rs, .txt.
- Guardar: descarga el contenido actual del editor en un archivo de texto.



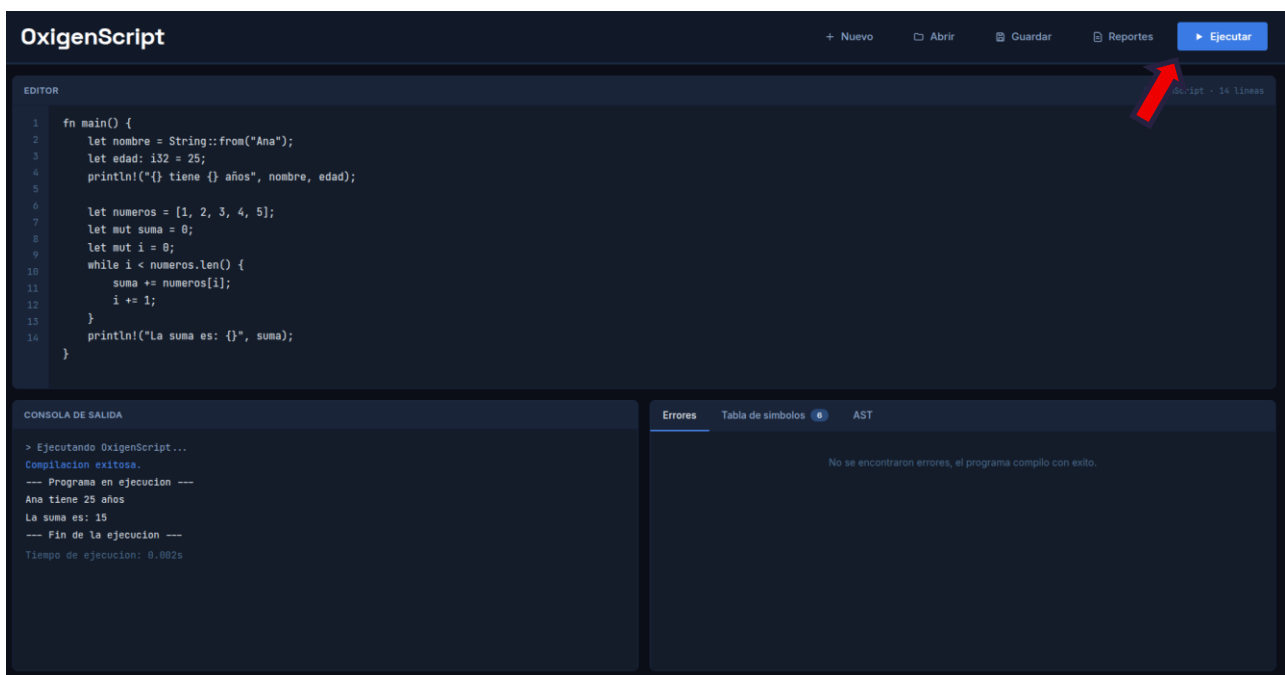
EJECUTAR PROGRAMA

Para ejecutar el contenido del editor de código debe seguir los siguientes pasos:

1. Escriba o pega el código en el editor.
2. Presione el botón *Ejecutar*.
3. El resultado aparecerá en la *Consola de salida*:

Programa compilado

Si el programa compila y corre sin problemas se mostrará un mensaje de *Compilación exitosa*, seguido de la salida del programa (lo impreso con `println!`) y el tiempo de ejecución.



REPORTE DE ERRORES

Cada fila de la tabla de errores se muestra de la siguiente forma:

Columna	Significado
Tipo	Léxico , Sintáctico o Semántico — en qué fase del análisis se detectó
Línea	Línea del código fuente donde ocurrió
Columna	Posición dentro de esa línea (cuando aplica)
Descripción	Explicación del problema

El intérprete no se detiene en el primer error. Sigue analizando el resto del archivo para reportar la mayor cantidad de problemas posibles en una sola ejecución (con la excepción de un error sintáctico muy grave puede impedir parseando el resto de una sentencia puntual).

Error típico

El siguiente código es un ejemplo:

```
rust
fn main() {
    let x = 10      // falta el punto y coma
    println!("{}", x);
}
```

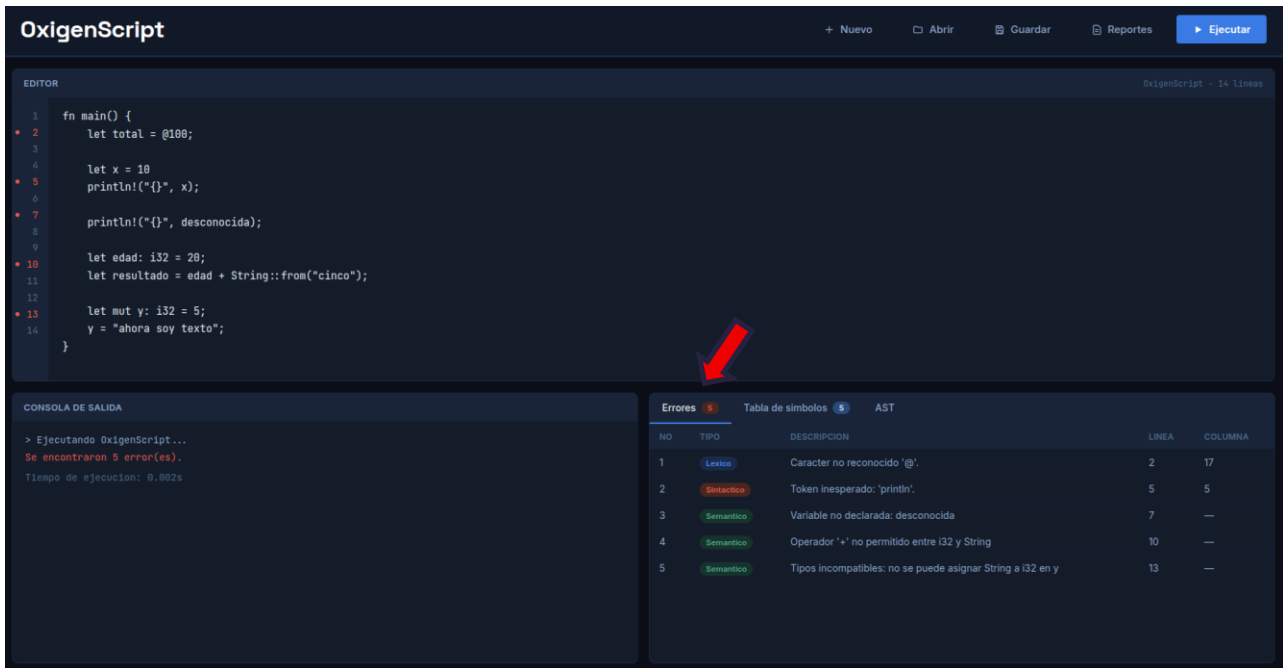
Reporta:

```
[Error Sintáctico] Línea 2, Columna 15
Token inesperado: 'println'.
```

Corrección: agregar ";" al final de la línea 2.

Programa compilado con errores

Si hay errores, la consola indicará cuántos se encontraron y el detalle completo se mostrará en la sección de *Errores*, fue posible mostrar el resultado de igual forma debido a la resiliencia.



The screenshot shows the OxygenScript IDE interface. The top bar includes a title 'OxygenScript' and buttons for '+ Nuevo', 'Abrir', 'Guardar', 'Reportes', and 'Ejecutar'. The main editor area displays a code snippet with 14 lines. The bottom panel is divided into three sections: 'CONSOLA DE SALIDA', 'Errores', and 'Tabla de símbolos'. The 'Errores' section is active, showing a table of 5 errors. A red arrow points to the 'Errores' tab.

EDITOR

```
1 fn main() {  
2   let total = @100;  
3  
4   let x = 10  
5   println!("{}", x);  
6  
7   println!("{}", desconocida);  
8  
9   let edad: i32 = 20;  
10  let resultado = edad + String::from("cinco");  
11  
12  let mut y: i32 = 5;  
13  y = "ahora soy texto";  
14 }
```

CONSOLA DE SALIDA

```
> Ejecutando OxygenScript...  
Se encontraron 5 error(es).  
Tiempo de ejecución: 0.002s
```

Errores

NO	TIPO	DESCRIPCION	LINEA	COLUMNA
1	Lexico	Caracter no reconocido '@'.	2	17
2	Sintactico	Token inesperado: 'println'.	5	5
3	Semantico	Variable no declarada: desconocida	7	—
4	Semantico	Operador '+' no permitido entre i32 y String	10	—
5	Semantico	Tipos incompatibles: no se puede asignar String a i32 en y	13	—

TABLA DE SÍMBOLOS

Esta tabla muestra solamente los identificadores definidos por el programa (variables, funciones, structs y parámetros) y no las palabras reservadas (fn, let, etc), ni los operadores o delimitadores ya que esos los maneja internamente el analizador léxico y no son símbolos que deban rastrearse semánticamente.

Columna	Significado
No.	Número consecutivo de la fila
Identificador	Nombre de la variable, función, parámetro o struct
Categoría	Variable, Función o Struct
Tipo	Tipo de dato (i32, bool, String, nombre del struct, etc.)
Ámbito	Dónde está declarado: global, o el nombre de la función que lo contiene
Línea	Línea donde se declaró
Valor	La expresión con la que se inicializó (no el resultado evaluado en tiempo de ejecución)



Shadowing

Si se declara dos veces "let" con el mismo nombre en el mismo ámbito, se mostrarán dos filas con ese identificador, una por cada declaración para dejar constancia de ambas.

¿Qué pasa si el código tiene errores?

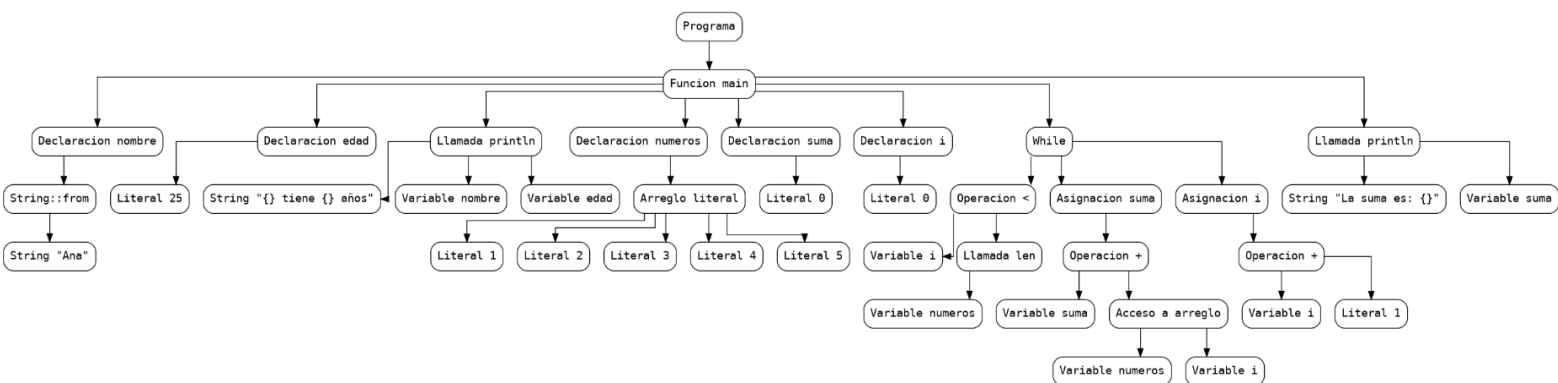
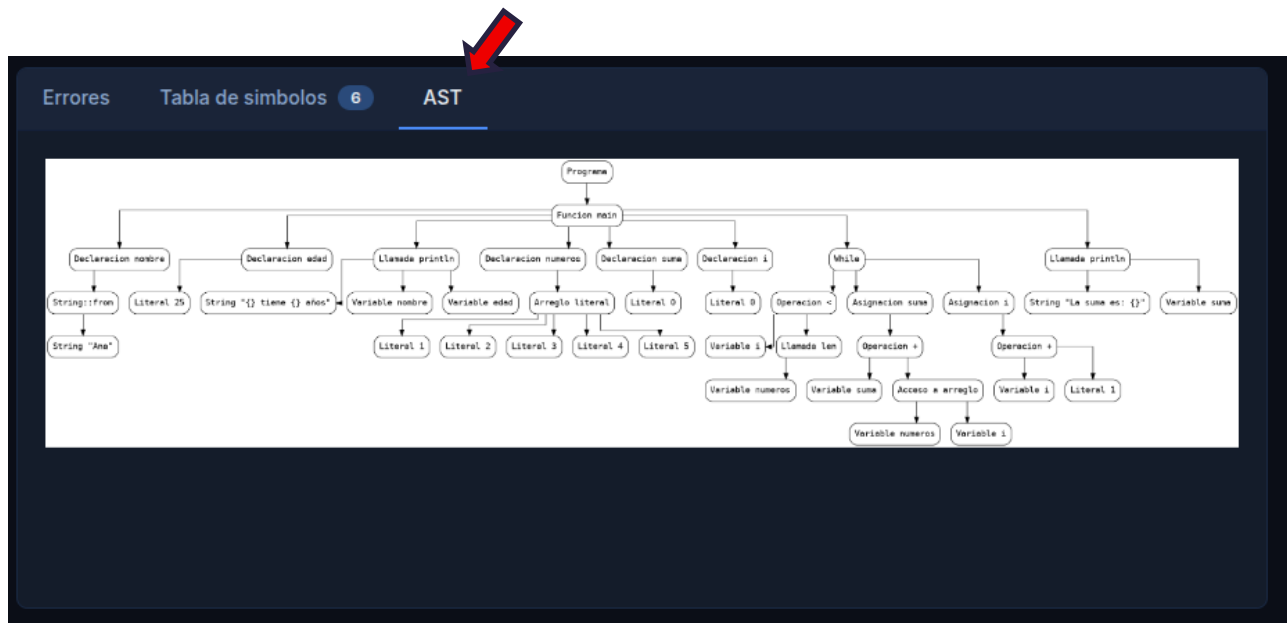
A pesar de que el código ejecutado tenga errores, se seguirá mostrando la tabla de símbolos de lo que se haya podido encontrar.

Errores	5	Tabla de símbolos	5	AST		
NO	IDENTIFICADOR	CATEGORIA	TIPO	AMBITO	LINEA	VALOR
1	main	Funcion	—	global	1	—
2	total	Variable	i32	main	2	100
3	edad	Variable	i32	main	9	20
4	resultado	Variable	—	main	10	edad + "cinco"
5	y	Variable	i32	main	12	5

ÁRBOL DE SINTAXIS ABSTRACTA

El AST se muestra en forma de grafo generado con Graphviz, donde cada nodo representa una construcción del lenguaje (una declaración, operación, llamada) y las flechas indican la relación padre-hijo entre ellas. Es útil para verificar que el programa se haya interpretado con la estructura y precedencia esperada, en especial al depurar expresiones aritméticas o lógicas complejas.

La imagen se muestra en el apartado del AST, en el caso de que sea muy grande se recomienda abrirlo en otra pestaña para visualizarlo de mejor manera.



PREGUNTAS FRECUENTES

¿Por qué el programa no imprime nada?

Asegúrese de revisar la pestaña de Errores, probablemente haya un error léxico, sintáctico o semántico el cual impide la ejecución completa.

¿Se puede usar for?

No, OxygenScript solo define `if`, `while`, `loop` y `match` como estructuras de control, según la especificación del lenguaje.

¿Cómo le pongo nombre a un loop para poder hacerle `break`/`continue` desde uno anidado?

Con una etiqueta antes del loop, con comilla simple: `'nombre: loop { ... }`, y luego `break 'nombre;` o `continue 'nombre;`.

¿Por qué un `String` con `\n` no se ve en dos líneas en la consola?

Depende de la configuración visual del navegador/CSS del proyecto, el valor interno sí contiene el salto de línea correctamente.